09. Feasify Performance, Load & Stress Testing Report

Document ID: FEASIFY-DOC-09

Test Suite: `feasify-backend/tests/comprehensiveQaAssessment.test.js`

Execution Timestamp: November 11, 2026 16:19:54 UTC

Auditor: Performance & Load Stress Testing Engineer

Status: Completed with Empirical Measurements

---

1. Concurrency Benchmark Results

Automated stress testing evaluated the backend HTTP engine under 5 graduated virtual user (VU) concurrency scenarios using high-resolution performance timers (`perf_hooks.performance`).

Scenario	Virtual Users (VUs)	Total Requests	Throughput (Req/Sec)	Average Latency	Median Latency	95th Percentile Latency	Error Rate	System Stability
Baseline	1 VU	2	**265.36**	3.74 ms	4.71 ms	4.71 ms	**0.0%**	Optimal
Low Traffic	10 VUs	20	**331.62**	26.57 ms	30.33 ms	45.45 ms	**0.0%**	Stable
Moderate Traffic	25 VUs	50	**404.23**	55.21 ms	58.22 ms	90.22 ms	**0.0%**	Stable
High Traffic	50 VUs	100	**410.97**	108.00 ms	121.49 ms	186.66 ms	**0.0%**	Minor queueing
Heavy Traffic	100 VUs	200	**417.12**	210.87 ms	246.36 ms	367.99 ms	**0.0%**	Sub-second latency

```
CONCURRENCY THROUGHPUT & LATENCY SCALING CURVE
Throughput (Req/s):
  1 VU   [████████████████] 265 req/s
 10 VUs  [████████████████████] 331 req/s
 25 VUs  [████████████████████████] 404 req/s
 50 VUs  [████████████████████████] 410 req/s
100 VUs  [█████████████████████████] 417 req/s

95th-Percentile Latency (p95):
  1 VU   4.7 ms
 10 VUs  45.5 ms
 25 VUs  90.2 ms
 50 VUs  186.7 ms
100 VUs  368.0 ms
```

---

2. In-Depth Performance Analysis

2.1 Backend Throughput & Concurrency Scaling

- **Throughput Saturation:** Throughput scales linearly from 265 req/s (1 VU) to 404 req/s (25 VUs) before plateauing at **417 req/s (100 VUs)**. This reflects the single-threaded Node.js event loop capacity on the host machine.
- **Latency Distribution:** Under 100 concurrent requests, the 95th percentile latency remains at **368 ms**, well below the 1,000 ms usability ceiling for interactive web requests.
- **Error Resilience:** Across all 372 executed stress test requests, the error rate was **0.0%**, confirming no unhandled socket disconnects or process crashes occurred.

2.2 Gemini AI LLM Latency & Cascade Architecture

- **Observed Characteristics:**
- Calls to Google Gemini (`gemini-3.5-flash-lite`) are external network operations with latencies typically ranging between **1.8 s and 5.5 s** depending on prompt token length.
- Feasify implements a multi-model cascade with an AbortController hard timeout of **75 seconds** (`feasify-backend/index.js` L270) and exponential backoff retry (`callGeminiWithRetry` , L295).
- Client UI properly manages this duration via animated progress indicators in `AIAnalysisLoading.tsx` to prevent student abandonment.

---

3. Frontend Bundle & Asset Size Performance

Extracted directly from Vite 7.3.2 production build logs:

Generated Asset	Minified Size	Gzip Compressed Size	Status / Evaluation
`dist/index.html`	0.47 kB	0.30 kB	Optimal
`dist/assets/index.css`	168.39 kB	29.86 kB	Clean Tailwind CSS purge
`dist/assets/purify.es.js`	24.10 kB	9.05 kB	Optimal
`dist/assets/index.es.js`	158.79 kB	53.02 kB	Vendor chunk split
`dist/assets/html2canvas.esm.js`	201.04 kB	47.43 kB	PDF export utility
`dist/assets/index.js` (Main Chunk)	**3,330.32 kB**	**930.19 kB**	**Warning: Chunk > 500 kB**

Bottleneck Finding PERF-01: Oversized Main Bundle Chunk

- **Finding:** The primary JavaScript bundle chunk is **3.33 MB (930 kB gzip)**, triggering Vite's build warning.
- **Cause:** Large vendor dependencies (`firebase` , `lucide-react` , `jspdf` , `xlsx` , `leaflet` , `recharts`) are compiled into a monolithic bundle without route-level lazy loading (`React.lazy()`).
- **Recommended Optimization:**

1. Implement code-splitting using `React.lazy()` for heavy modules (`ChairpersonFeasib` , `Financial_input` , `AI_Analysis`).

2. Configure `build.rollupOptions.output.manualChunks` in `vite.config.ts` to separate Firebase, Recharts, and SheetJS into vendor chunks.